

UDESC

Departamento de Ciência da Computação

Estrutura de Dados 2

**Análise comparativa da complexidade algorítmica de
árvores AVL, rubro-negras e B**

Alunos: Gabriela Alves Ilezyszyn, Heitor Linhares Caetano e Pedro Henrique Dias
Professor: Allan Rodrigo Leite

Junho
2026

Conteúdo

1	Objetivo do Trabalho	1
2	Metodologia	2
2.1	Contagem de comparações	2
3	Implementação	3
3.1	Árvore AVL	3
3.2	Árvore Rubro-negra	3
3.3	Árvores B	3
4	Resultados	4
4.1	Análise dos resultados	4
5	Considerações Finais	5

1 Objetivo do Trabalho

O objetivo do trabalho é analisar e comparar a complexidade computacional das operações de inserção e remoção de nós em árvores AVL, rubro-negras e B, considerando também o custo de balanceamento. Para isso, foram usados conjuntos de dados aleatórios com tamanhos variando de 1 a 10 mil. Os resultados foram apresentados em gráficos que relacionam o esforço temporal das operações, seguidos de uma discussão sobre o comportamento e a eficiência das diferentes estruturas apresentadas.

2 Metodologia

Para a criação dos conjuntos aleatórios, foi criado um vetor de tamanho 10 mil em que cada posição armazena o próprio índice. Após isso, é usado um [algoritmo de embaralhamento](#) que randomiza o vetor com base em valores aleatórios obtidos com a função *rand*.

A partir desse vetor, o experimento foi construído de forma a avaliar o crescimento das estruturas de maneira *contínua*. Ou seja, as árvores não são destruídas e recriadas a cada ciclo, mas sim crescem um tamanho de passo (400) a cada iteração, chegando a 10.000 na última.

Quando o experimento chega em um dos pontos de amostragem (um múltiplo do tamanho do passo), o esforço computacional das operações é registrado. Para a inserção, é registrado apenas o custo das últimas 400 chaves (janela entre o passo analisado e o próximo). Isso faz com que o comportamento seja isolado e consigamos coletar apenas o custo da parte que importa, sem ser "diluído" pelo custo das inserções anteriores.

Já para a remoção, no mesmo ponto, primeiramente são criadas cópias temporárias das árvores que estamos criando. Após isso, os elementos delas são copiados para um vetor auxiliar, embaralhados novamente para que tenhamos uma ordem de remoção aleatória, e aí sim são removidos da estrutura temporária, com as comparações sendo contabilizadas. Isso faz com que possamos contabilizar o custo de remoção em determinado tamanho sem interromper o crescimento das árvores principais.

2.1 Contagem de comparações

Para estabelecer um método de comparação entre a complexidade das diferentes árvores, se faz necessário o uso da comparação como unidade de esforço computacional. Comparações são todas as partes do código-fonte que estabelecem uma mudança de fluxo condicional ao sistema. Na linguagem C, elas são invocadas em toda aparição das *keywords* *if*, *for*, *while*, *do{..}while*, além do operador ternário *?*.

Contudo, nem toda ocorrência desses termos é contabilizada para os fins deste trabalho. A justificativa para isso é de que nem toda comparação no código é inerente à estrutura analisada, sendo muitas vezes apenas parte da organização do fluxo do programa. O principal exemplo de comparação não contabilizada em nosso código são as variáveis e condições de laços de repetição, como *j > i*, operadores de verificação de ponteiros, como *if(raiz == NULL)* ou *if(!raiz)*, e validações de alocações de memória.

Essas operações são produtos apenas de decisões de implementação e

gerenciamento de fluxo em linguagem C, e não representam o custo "matemático" do algoritmo. Portanto, foram contabilizados apenas os testes de relação entre chaves e de elementos armazenados, esse sim próprios a cada estrutura de dados.

3 Implementação

3.1 Árvore AVL

A árvore AVL foi implementada de maneira similar a demonstrada em aula, porém com uma pequena diferença: cada nó também armazena um inteiro que representa sua altura. Essa altura é atualizada a cada inserção ou remoção e é usada no cálculo do fator de balanceamento, que se torna muito mais simples com essa informação já em mãos, sendo apenas a diferença entre as alturas das subárvores esquerda e direita ($FB = altura(esquerda) - altura(direita)$). Caso o valor absoluto do fator de balanceamento atinja 2, as rotações simples ou duplas são acionadas para reestruturar a árvore.

Esse armazenamento das alturas de cada nó ajuda significativamente na eficiência da árvore, sendo o padrão na maioria das implementações disponíveis na *internet*. Se a estrutura precisasse percorrer recursivamente por todas suas subárvores para descobrir suas alturas a cada inserção ou remoção, a operação de balanceamento teria maior complexidade, sendo $O(n)$ no pior dos casos, diminuindo a vantagem de se usar uma árvore auto-balanceada.

Quando cada nó armazena explicitamente sua altura, o cálculo do fator de balanceamento se torna uma operação de esforço constante ($O(1)$).

3.2 Árvore Rubro-negra

3.3 Árvores B

4 Resultados

4.1 Análise dos resultados

5 Considerações Finais